3.1 System Architecture

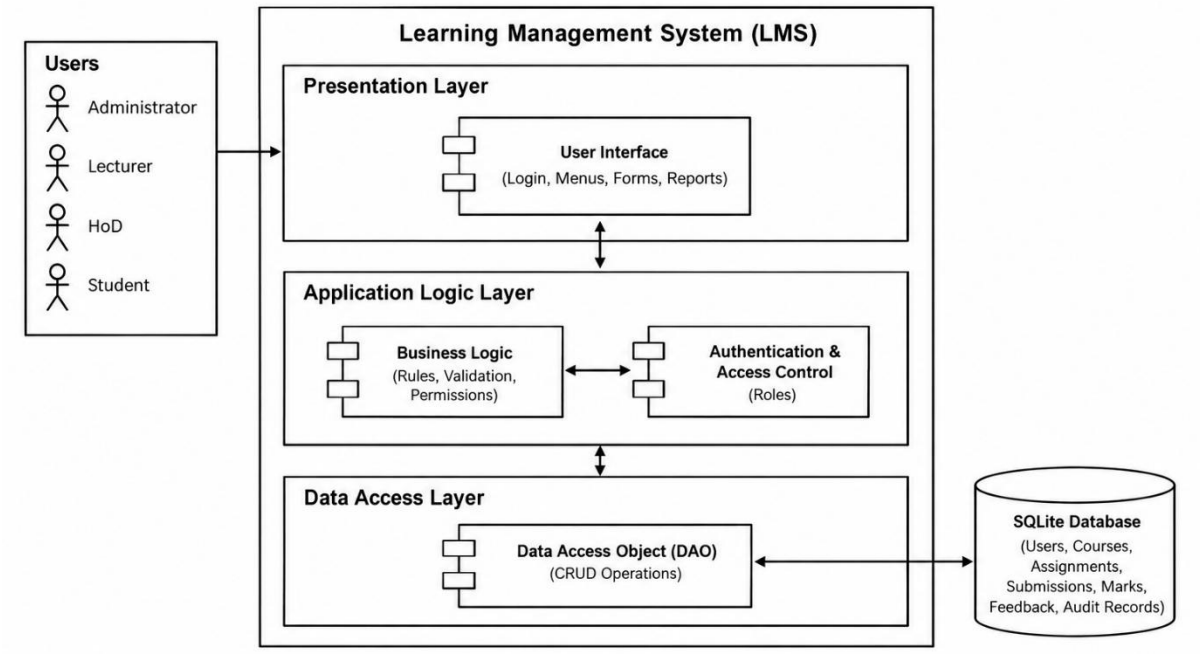
3.1.1 Architectural Style

The Learning Management System (LMS) will use a Layered Architecture. This architecture separates the system into three layers: the Presentation Layer, Application Logic Layer and Data Access Layer. Each layer has its own responsibility, which makes the system easier to develop, test and maintain.

The Presentation Layer will provide the interface for the Administrator, Lecturer, Head of Department (HoD) and Student. The Application Logic Layer will handle the system's rules and operations, including authentication, role permissions, assignment submissions, late penalties, marking and moderation. The Data Access Layer will communicate with the SQLite database to store and retrieve information.

This architecture is suitable for the project because the LMS must operate offline, use C++17 and SQLite, and protect student and assessment records. Separating the layers will also make it easier to test individual parts of the system before combining them.

3.1.2 Component Diagram



The component diagram shows the main software components and how they communicate. User requests pass through the presentation components to the application logic, which applies the required rules before accessing the database through the data access components. The SQLite database stores the system's users, courses, assignments, submissions, marks, feedback and audit records.

3.1.3 Design Rationale

The Layered Architecture was selected because it separates user interaction, business rules and data storage. This separation makes the system easier to understand and maintain. For example, changes to the user interface can be made without unnecessarily changing how marks are stored in SQLite.

The architecture also supports testing. Authentication, submission handling, marking and moderation logic can be tested separately from the database and user interface. This will help identify problems before the system is deployed.

The design supports offline operation because the application and database can run on the local computer without requiring an internet connection. It also provides a clear place to enforce role-based permissions and ensure that marks cannot be released before the HoD has approved them. A Model–View–Controller (MVC) architecture was considered as an alternative. MVC separates the data model, user interface and controller that handles user actions. It can be useful for applications with complex interfaces that require frequent changes.

However, the proposed LMS is a relatively small offline application with clearly defined functions and a local SQLite database. A three-layer architecture provides a simpler separation between the interface, business logic and data access without requiring a separate MVC structure.

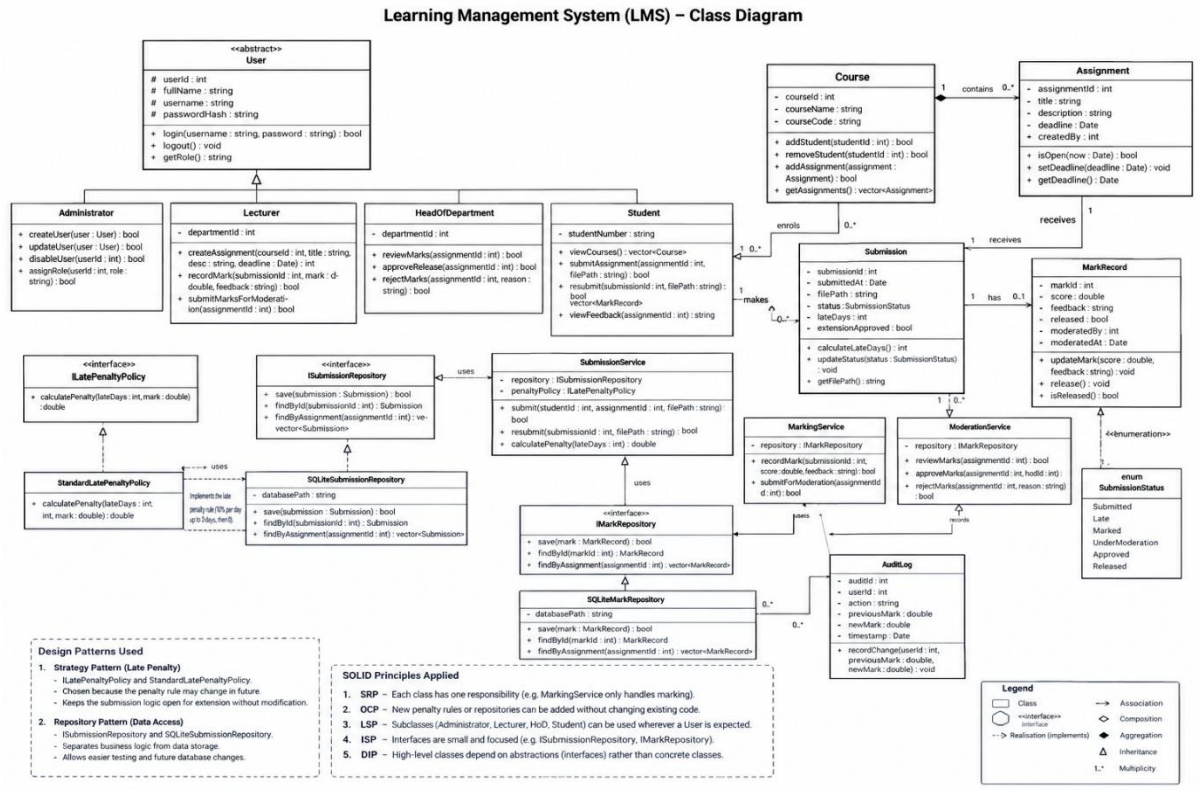
Therefore, the Layered Architecture was selected because it is suitable for the project's size, required technologies and seven-week development schedule.

3.2 Class Diagram

3.2.1 Introduction

The class diagram shows the static structure of the Learning Management System (LMS), including its classes, attributes, operations and relationships. It represents the design that will guide implementation using C++17 and SQLite.

3.2.2 Class Diagram



3.2.3 Design Patterns Applied

1. Strategy Pattern

The Strategy Pattern is used for late-submission penalties through the ILatePenaltyPolicy interface and StandardLatePenaltyPolicy class. The SubmissionService uses this policy to calculate penalties without directly depending on the penalty calculation.

This makes it easier to change the late-submission policy without rewriting the submission process. Under the current policy, submissions up to three days late receive a 10% penalty per day, while submissions later than three days receive zero unless an approved extension applies.

2. Repository Pattern

The Repository Pattern is used through the ISubmissionRepository interface and SQLiteSubmissionRepository class. The interface defines how submissions are saved and retrieved, while the SQLite implementation handles database operations.

This separates business logic from data storage, making the system easier to test and maintain. The database implementation can also be changed without unnecessarily changing the submission service.

3.2.4 Application of SOLID Principles

- **Single Responsibility Principle (SRP):** Each class has a specific responsibility. For example, SubmissionService handles submission operations, while MarkingService handles marking and ModerationService handles moderation.
- **Open/Closed Principle (OCP):** New late-penalty policies can be introduced by implementing ILatePenaltyPolicy without rewriting the submission service.
- **Liskov Substitution Principle (LSP):** The concrete user-role classes can be used wherever the general User type is expected, while preserving the expected behaviour of the base class.
- **Interface Segregation Principle (ISP):** The submission repository interface contains submission-related operations instead of forcing every class to implement unrelated database functions.
- **Dependency Inversion Principle (DIP):** SubmissionService depends on the ILatePenaltyPolicy and ISubmissionRepository abstractions rather than depending directly on concrete implementations.

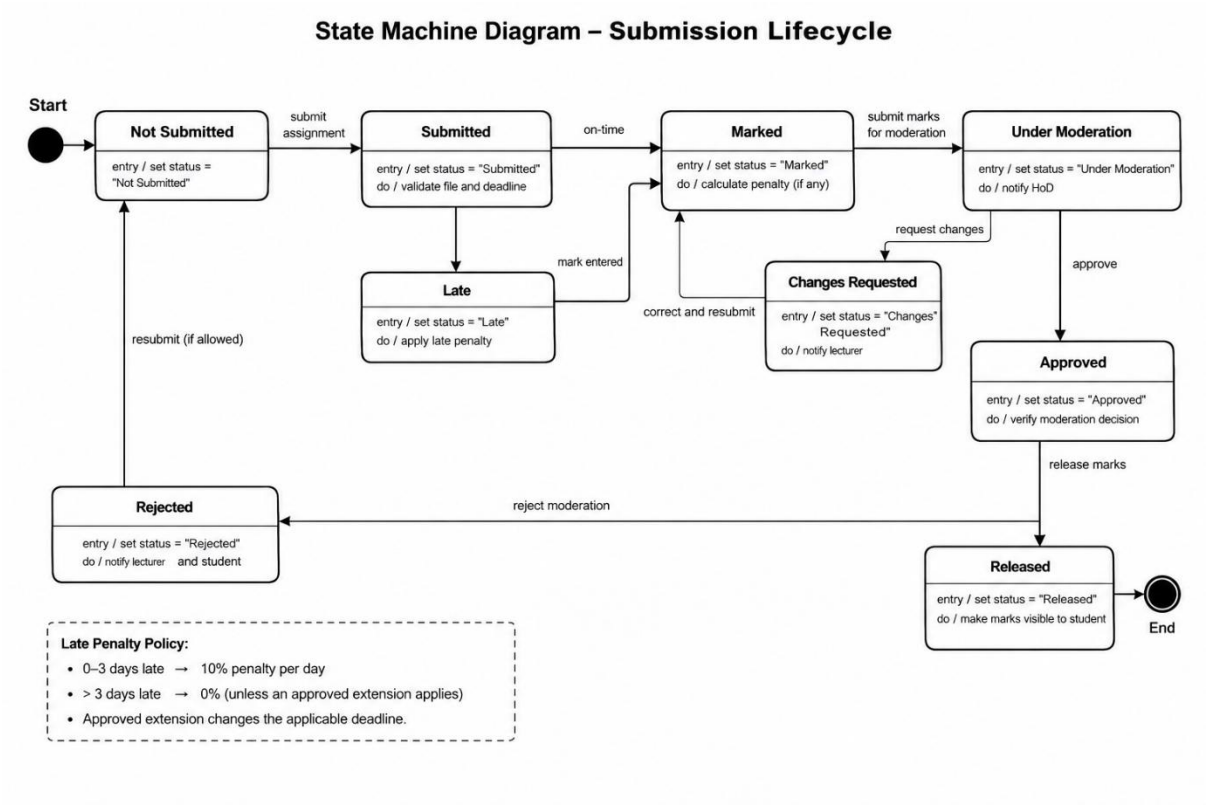
3.3 Behavioural Models

3.3.1 Introduction

Behavioural models describe how the Learning Management System (LMS) behaves during its operation. They show the lifecycle of a submission, the activities performed by each user, and the interactions between system components. The models cover assignment submission, late penalties, resubmission, marking, moderation and release of marks.

3.3.2 State Machine Diagram — Submission Lifecycle

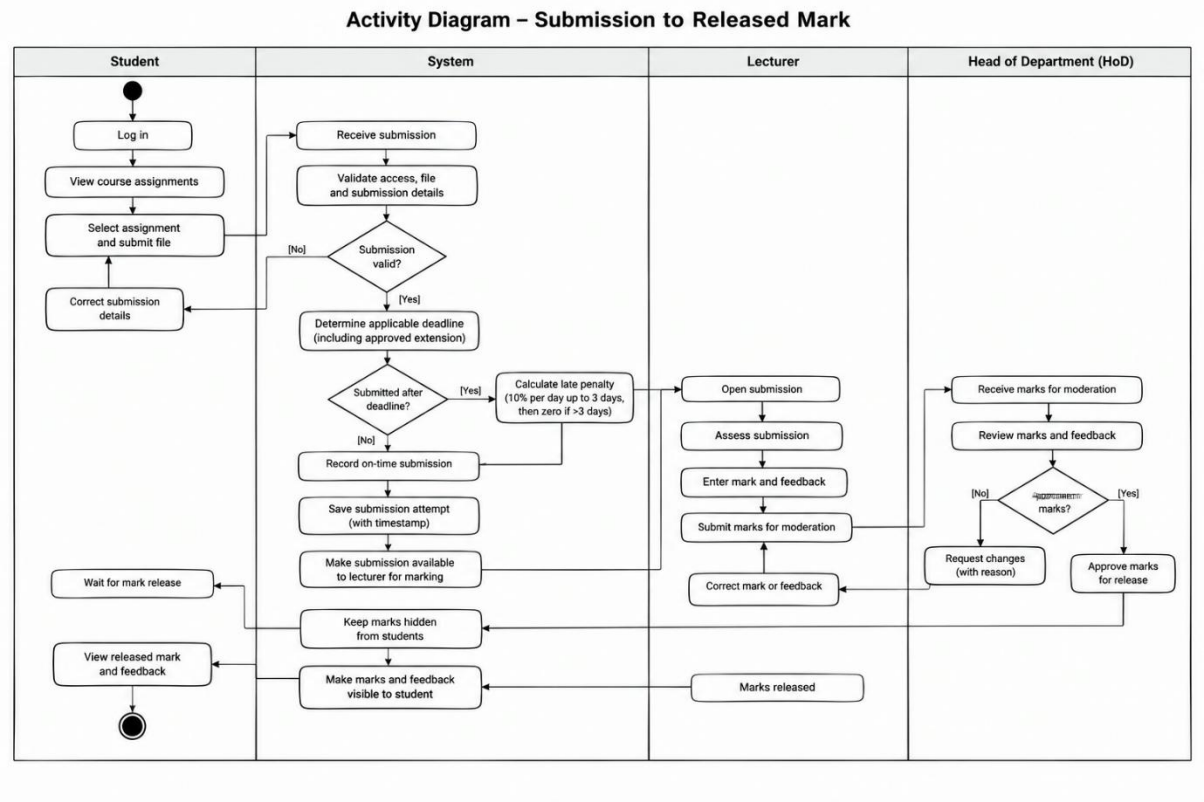
The state machine diagram shows the different states of a submission and the events that cause it to move from one state to another. It includes submission, marking, moderation, requested changes, approval and release. It also shows that marks cannot be released before approval by the Head of Department (HoD).



The diagram shows that a submission moves to the marking stage after it has been received. Late submissions are subject to the applicable penalty rules. After marking, the lecturer submits the marks for moderation. If the HoD requests changes, the lecturer must correct the marks and submit them for review again. Marks are released only after approval.

3.3.3 Activity Diagram — Submission to Released Mark

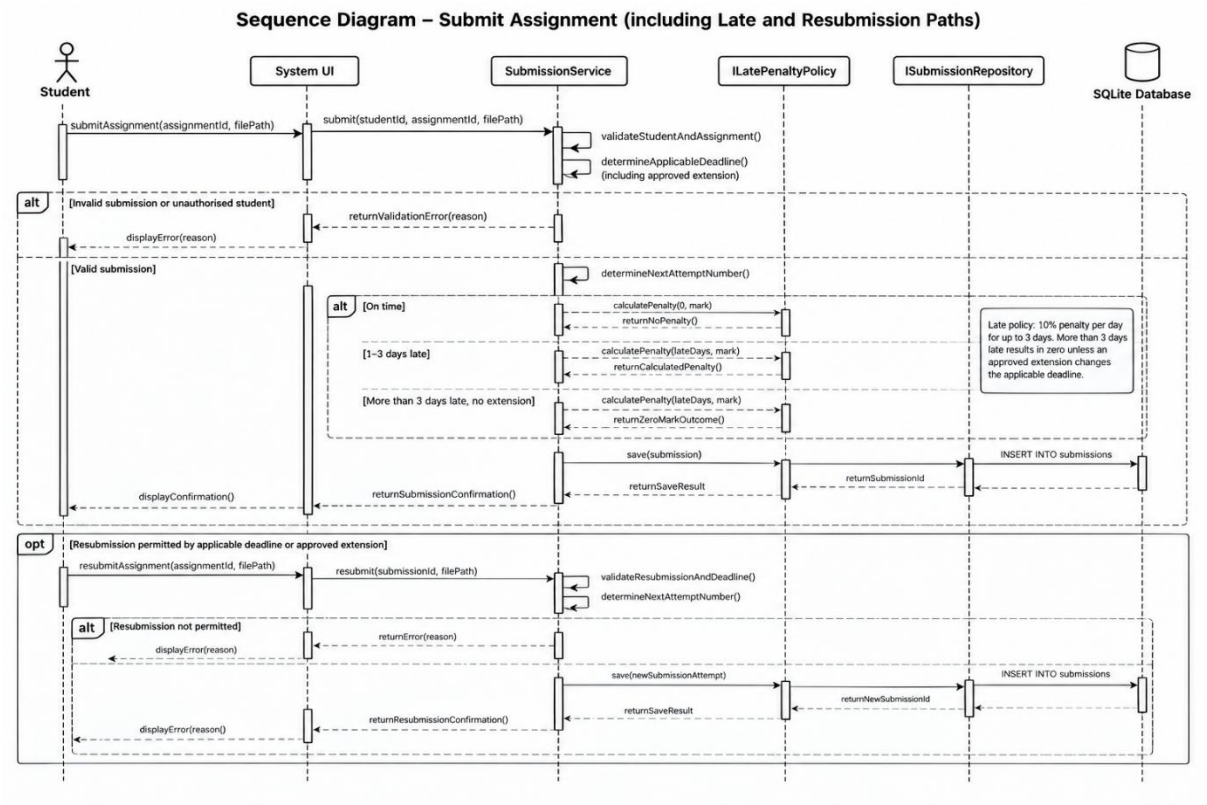
The activity diagram represents the complete process from assignment submission to the release of marks. Swimlanes identify the responsibilities of the Student, System, Lecturer and HoD. Decision points show how the system handles invalid submissions, late work, extensions and moderation outcomes.



The process begins when a student logs in, selects an assignment and submits a file. The system validates the submission, checks the applicable deadline and records the attempt. The lecturer then marks the submission and sends the marks for moderation. The HoD either approves the marks or requests corrections. Once approval is obtained, the system releases the marks and feedback to the student.

3.3.4 Sequence Diagram — Submit Assignment

The sequence diagram shows the messages exchanged between the student, system interface, SubmissionService, ILatePenaltyPolicy, ISubmissionRepository and SQLite database during submission. It includes the normal submission path, the late-submission path and the resubmission path.

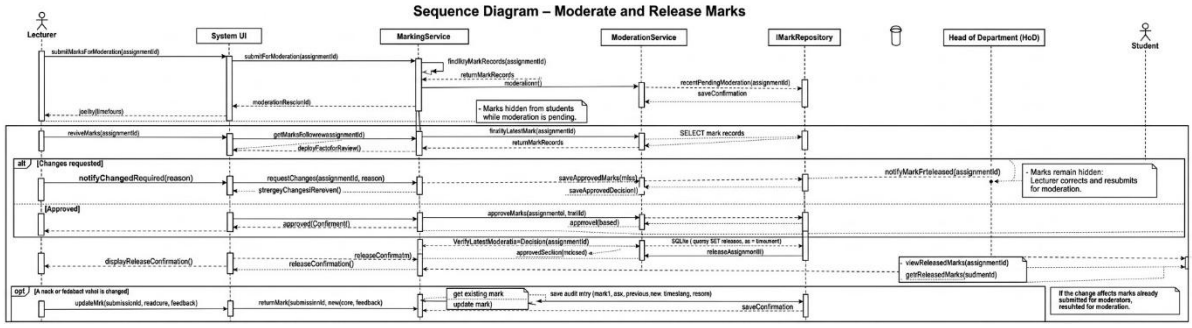


The student submits an assignment through the system interface. The SubmissionService validates the submission and determines the applicable deadline, including any approved extension. The penalty policy calculates the applicable late penalty, and the repository saves the submission attempt in the database. A resubmission is stored as a separate attempt so that the previous submission history is preserved.

Under the agreed policy, late submissions receive a 10% penalty per day for up to three days. Submissions more than three days late receive zero unless an approved extension changes the applicable deadline.

3.3.5 Sequence Diagram — Moderate and Release Marks

This sequence diagram shows how marks move from lecturer marking to HoD moderation and eventual release. It illustrates the interaction between the lecturer, HoD, system interface, MarkingService, ModerationService, repository and SQLite database.



The lecturer records marks and feedback and submits them for moderation. The HoD reviews the marks and either approves them or requests changes. When changes are requested, the lecturer corrects the marks and resubmits them for moderation. When the HoD approves the marks, the

system records the decision and releases the marks to students. Mark changes are recorded in the audit log to support accountability.

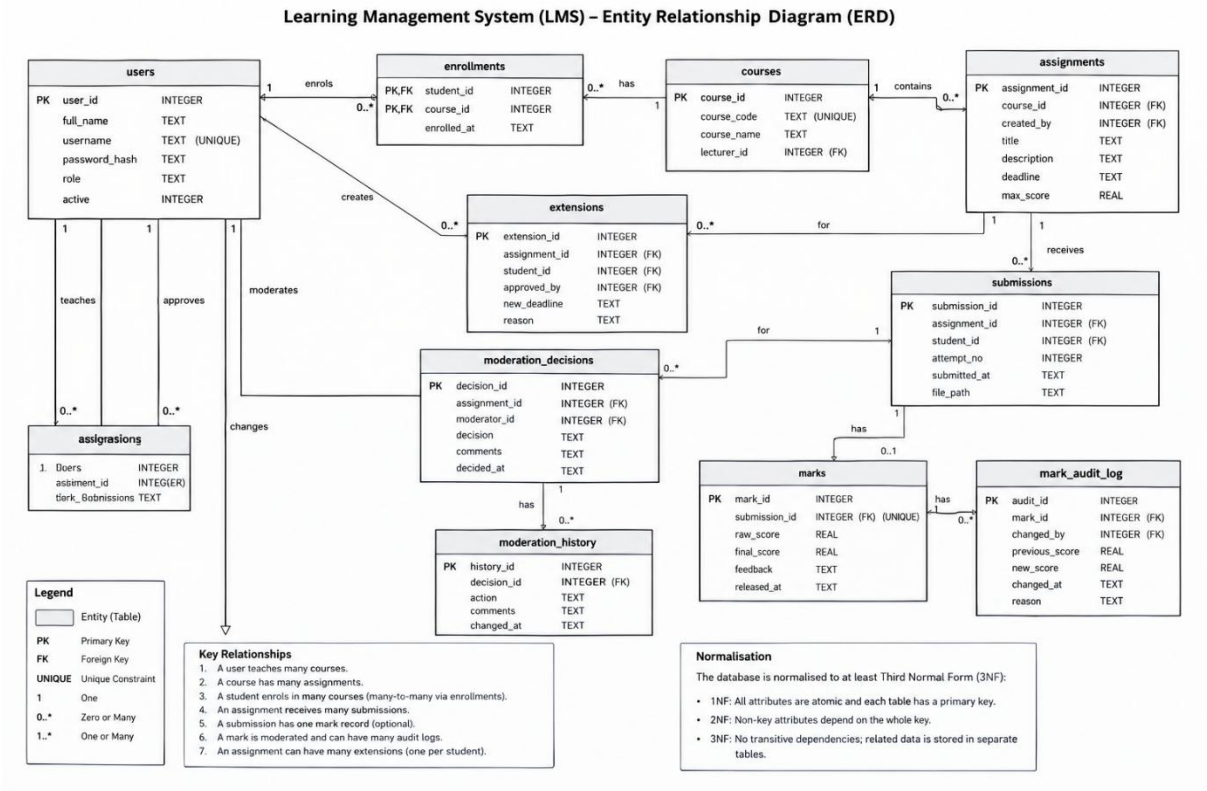
3.4 Database Design

3.4.1 Introduction

The LMS database will use SQLite to store user accounts, courses, assignments, submissions, marks, moderation decisions and audit records. The design is normalised to Third Normal Form (3NF) to reduce data duplication and maintain data integrity.

Primary keys uniquely identify records, foreign keys link related tables, and constraints prevent invalid data. The database also supports resubmissions, approved extensions and the rule that marks must be moderated before students can view them.

3.4.2 Entity Relationship Diagram (ERD)



The ERD contains the main entities and their relationships. A student can enrol in many courses, and each course can have many students. A course can contain several assignments. Students may make multiple submissions for an assignment, while each submission can have a mark record. Approved extensions, moderation decisions and mark changes are stored separately.

3.4.3 Normalisation to Third Normal Form (3NF)

The database is designed to satisfy the following normalisation rules:

First Normal Form (1NF): Each field stores a single value, and each table has a primary key. For example, course enrolments are stored as separate rows in enrollments, rather than storing several student numbers in one course record.

Second Normal Form (2NF): The database avoids partial dependencies on part of a composite key. In enrollments, the composite primary key is (student_id, course_id), and the table contains no unrelated student or course details. Those details are stored in users and courses.

Third Normal Form (3NF): Non-key attributes depend on the key, the whole key and nothing but the key. Student details are stored in users, course details in courses, assignment details in assignments, and moderation decisions in moderation_decisions. This avoids storing the same user, course or moderation information repeatedly in unrelated records.

The number of late days and the late-penalty amount are calculated from the applicable deadline, submission time and penalty policy instead of being stored as duplicated derived values. This reduces inconsistency when deadlines or approved extensions change.

3.4.4 Data Dictionary

The following dictionary describes the main database tables. PK means Primary Key, FK means Foreign Key, UQ means Unique, and NN means Not Null.

Table	Important fields and purpose
users	user_id (PK); full_name, username (UQ), password_hash, role, active. Stores all four user roles.
courses	course_id (PK); course_code (UQ), course_name, lecturer_id (FK). Stores course details and the responsible lecturer.
enrollments	student_id (PK, FK), course_id (PK, FK), enrolled_at. Links students to courses and prevents duplicate enrolments.
assignments	assignment_id (PK); course_id (FK), created_by (FK), title, description, deadline, max_score. Stores assessment details.
extensions	extension_id (PK); assignment_id (FK), student_id (FK), approved_by (FK), new_deadline, reason. Stores individual approved extensions.
submissions	submission_id (PK); assignment_id (FK), student_id (FK), attempt_no, submitted_at, file_path. Stores each attempt separately.
marks	mark_id (PK); submission_id (FK, UQ), raw_score, final_score, feedback, released_at. Stores the mark and feedback for a submission.
moderation_decisions	decision_id (PK); assignment_id (FK), moderator_id (FK), decision, comments, decided_at. Stores HoD approval or requests for changes.
mark_audit_log	audit_id (PK); mark_id (FK), changed_by (FK), previous_score, new_score, changed_at, reason. Records mark changes for accountability.

Important constraints: usernames and course codes must be unique; each student can enrol in a particular course only once; each submission attempt number must be unique for a student and assignment; each submission has at most one current mark record; scores must be within the permitted range; and a student must not receive marks for an assignment in a course in which they are not enrolled.

3.4.5 SQL Schema

The following SQLite schema defines the tables, primary keys, foreign keys and main constraints. Foreign-key enforcement must be enabled for each database connection.

```
PRAGMA foreign_keys = ON;


CREATE TABLE users (
   user_id INTEGER PRIMARY KEY,
   full_name TEXT NOT NULL,
   username TEXT NOT NULL UNIQUE,
   password_hash TEXT NOT NULL,
   role TEXT NOT NULL CHECK (
      role IN (
         'Administrator',
         'Lecturer',
         'HeadOfDepartment',
         'Student'
      )
   ),
   active INTEGER NOT NULL DEFAULT 1
      CHECK (active IN (0, 1))
);

CREATE TABLE courses (
   course_id INTEGER PRIMARY KEY,
   course_code TEXT NOT NULL UNIQUE,
   course_name TEXT NOT NULL,
   lecturer_id INTEGER NOT NULL,
   FOREIGN KEY (lecturer_id)
      REFERENCES users(user_id)
);

CREATE TABLE enrollments (
   student_id INTEGER NOT NULL,
   course_id INTEGER NOT NULL,
```

```
    enrolled_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES users(user_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

CREATE TABLE assignments (
    assignment_id INTEGER PRIMARY KEY,
    course_id INTEGER NOT NULL,
    created_by INTEGER NOT NULL,
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    deadline TEXT NOT NULL,
    max_score REAL NOT NULL CHECK (max_score > 0),
    FOREIGN KEY (course_id) REFERENCES courses(course_id),
    FOREIGN KEY (created_by) REFERENCES users(user_id)
);

CREATE TABLE extensions (
    extension_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    approved_by INTEGER NOT NULL,
    new_deadline TEXT NOT NULL,
    reason TEXT NOT NULL,
    FOREIGN KEY (assignment_id)
        REFERENCES assignments(assignment_id),
    FOREIGN KEY (student_id)
        REFERENCES users(user_id),
    FOREIGN KEY (approved_by)
        REFERENCES users(user_id),
    UNIQUE (assignment_id, student_id)
);
```

```
CREATE TABLE submissions (
    submission_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    attempt_no INTEGER NOT NULL CHECK (attempt_no >= 1),
    submitted_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    file_path TEXT NOT NULL,
    FOREIGN KEY (assignment_id)
        REFERENCES assignments(assignment_id),
    FOREIGN KEY (student_id)
        REFERENCES users(user_id),
    UNIQUE (assignment_id, student_id, attempt_no)
);

CREATE TABLE marks (
    mark_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL UNIQUE,
    raw_score REAL NOT NULL CHECK (raw_score >= 0),
    final_score REAL NOT NULL CHECK (final_score >= 0),
    feedback TEXT NOT NULL DEFAULT '',
    released_at TEXT,
    FOREIGN KEY (submission_id)
        REFERENCES submissions(submission_id)
);

CREATE TABLE moderation_decisions (
    decision_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    moderator_id INTEGER NOT NULL,
    decision TEXT NOT NULL CHECK (
        decision IN ('Approved', 'ChangesRequested')
    ),
    comments TEXT NOT NULL DEFAULT '',
    decided_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (assignment_id)
        REFERENCES assignments(assignment_id),
    FOREIGN KEY (moderator_id)
        REFERENCES users(user_id)
);

CREATE TABLE mark_audit_log (
    audit_id INTEGER PRIMARY KEY,
    mark_id INTEGER NOT NULL,
    changed_by INTEGER NOT NULL,
    previous_score REAL,
    new_score REAL NOT NULL CHECK (new_score >= 0),
    changed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    reason TEXT NOT NULL DEFAULT '',
    FOREIGN KEY (mark_id) REFERENCES marks(mark_id),
    FOREIGN KEY (changed_by) REFERENCES users(user_id)
);

-- Ensure scores do not exceed the assignment maximum.
CREATE TRIGGER validate_mark_insert
BEFORE INSERT ON marks
FOR EACH ROW
WHEN NEW.raw_score > (
    SELECT a.max_score
    FROM submissions s
    JOIN assignments a
      ON a.assignment_id = s.assignment_id
    WHERE s.submission_id = NEW.submission_id
)
OR NEW.final_score > (
    SELECT a.max_score
    FROM submissions s
    JOIN assignments a
      ON a.assignment_id = s.assignment_id
```

```
    WHERE s.submission_id = NEW.submission_id
)
BEGIN
    SELECT RAISE(ABORT, 'Mark exceeds assignment maximum');
END;

CREATE TRIGGER validate_mark_update
BEFORE UPDATE OF raw_score, final_score ON marks
FOR EACH ROW
WHEN NEW.raw_score > (
    SELECT a.max_score
    FROM submissions s
    JOIN assignments a
      ON a.assignment_id = s.assignment_id
    WHERE s.submission_id = NEW.submission_id
)
OR NEW.final_score > (
    SELECT a.max_score
    FROM submissions s
    JOIN assignments a
      ON a.assignment_id = s.assignment_id
    WHERE s.submission_id = NEW.submission_id
)
BEGIN
    SELECT RAISE(ABORT, 'Mark exceeds assignment maximum');
END;

-- A mark cannot be released without an approved moderation decision.
CREATE TRIGGER prevent_unapproved_mark_release
BEFORE UPDATE OF released_at ON marks
FOR EACH ROW
WHEN NEW.released_at IS NOT NULL
AND NOT EXISTS (
    SELECT 1
```

```
    FROM submissions s
    JOIN moderation_decisions d
      ON d.assignment_id = s.assignment_id
    WHERE s.submission_id = NEW.submission_id
      AND d.decision = 'Approved'
      AND d.decided_at <= NEW.released_at
      AND d.decided_at = (
        SELECT MAX(d2.decided_at)
        FROM moderation_decisions d2
        WHERE d2.assignment_id = s.assignment_id
      )
)
BEGIN
    SELECT RAISE(
      ABORT,
      'Marks cannot be released before moderation approval'
    );
END;
```

The application must also validate that only the assigned lecturer can create or mark assignments, only an authorised HoD can moderate marks, and only approved marks can be released to students. These role and workflow checks should be performed in the application, with database constraints and triggers providing additional protection where appropriate.

3.4.6 Mapping Tables to Classes

Database table	Corresponding class(es)	Explanation
users	User, Administrator, Lecturer, HeadOfDepartment, Student	One table stores shared user details and a role value. The C++ classes represent role-specific behaviour.
courses	Course	Stores each course and its responsible lecturer.
enrollments	Course, Student	Resolves the many-to-many relationship between courses and students.
assignments	Assignment, Course	Stores assignments belonging to a course.
extensions	Assignment, Student, HeadOfDepartment or authorised approver	Represents an approved deadline change for one student and assignment.

Database table	Corresponding class(es)	Explanation
submissions	Submission, SubmissionService	Stores each submission attempt; the service controls submission and resubmission rules.
marks	MarkRecord, MarkingService	Stores the mark and feedback associated with a submission.
moderation_decisions	ModerationService, HeadOfDepartment	Records the decision and comments for moderation.
mark_audit_log	AuditLog, MarkingService	Stores who changed a mark, when it changed, and the previous and new scores.

3.4.7 Justification of Differences Between the ERD and Class Diagram

The database and class diagram serve different purposes. The class diagram represents software objects and behaviour, while the ERD represents persistent data and its integrity rules.

The User inheritance hierarchy is stored in one users table with a role column. This avoids creating separate tables for common login details while allowing the C++ classes to provide different behaviour for each role.

The enrollments table is added to resolve the many-to-many relationship between students and courses. The extensions, moderation_decisions and mark_audit_log tables are separate because each can have its own history and must be recorded independently.

The repository interfaces and service classes are not stored as tables because they implement application logic and data access rather than represent persistent business records. The database stores the information those classes manage.